A PROJECT REPORT

ON

"EVENT MANAGEMENT SYSTEM"

Submitted by

MR. ARCHIT DUBEY
AND
SHIVAM JHA

Submitted In full fulfilment of the requirement for the degree

of

Bachelor of Science Information Technology

Under the guidance of

Mrs. LALTA NIRMAL

Department of Information Technology

Mahendar Pratap Sharada Prasad Singh College

of

Commerce and Science

(SEM-VI)

(2025-26)

Mahendra Pratap Sharada Prasad Singh College of Science, & Commerce (Affiliated to University of Mumbai)
(College Code - 729)

Department of Information Technology

CERTIFICATE

This is to certify that Mr. **Archit Dubey And Shivam Jha** of **T.Y.B.Sc.IT (Sem VI)** class has satisfactorily completed Project **EVENT MANAGEMENT SYSTEM** to be submitted in the full fulfillment for the award of **Bachelor of Science** in **Information Technology** during the academic year 2025 – 2026.

Date of Submission:

Project Guide

Head /In charge,
Department Computer Science

College Seal

Principal's Signature

Signature of Examiner

Abstract

In the modern era of digital transformation, traditional event planning methods that rely on manual record-keeping, fragmented communication, and unclear pricing structures have become inefficient. These outdated processes often lead to delays, miscommunication, and difficulties in coordinating event services. To overcome these challenges, a centralized and automated solution is required to simplify and streamline the event management process.

EventO is a full-stack web application designed to modernize event management by providing a single platform for both clients and event organizers. The system is developed using the MERN stack, which includes MongoDB, Express.js, React, and Node.js. This technology stack enables the application to deliver high performance, scalability, and a responsive user interface while efficiently managing event-related data.

The platform introduces a structured Tiered Service Model consisting of three packages: Silver, Gold, and Platinum. These packages allow users to choose event services based on their budget and requirements, ensuring transparency in pricing and service offerings. Each package provides different levels of features and support, making it easier for customers to select the most suitable option for their events.

Security and system integrity are maintained through a Role-Based Access Control (RBAC) mechanism implemented using JSON Web Tokens (JWT). This ensures secure authentication and authorization for different user roles within the system, including Users, Organizers, and Administrators. Each role has specific permissions, allowing the platform to provide a personalized, secure, and efficient event management experience.

ACKNOWLEDGEMENT

I am pleased to present the “EventO – Event Management System” project and take this opportunity to express my sincere gratitude to our Prof. LALTA NIRMAL for their constant guidance, motivation, and support in completing this project successfully.

No project can be completed without the valuable guidance of experienced mentors who help in shaping ideas and providing the right direction. I would like to take this opportunity to thank all the individuals who contributed directly or indirectly to the successful completion of this project.

I am thankful to my college for providing the necessary resources, facilities, and a supportive learning environment that helped me develop and present this project effectively.

I express my deepest gratitude to my project guide for their valuable suggestions, encouragement, and timely advice during the various phases of the project development. Their patience, support, and trust in my abilities played a significant role in the successful completion of this project.

Lastly, I would like to thank everyone who supported and encouraged me throughout the development of this project.

DECLARATION

We, the undersigned **Mr. ARCHIT DUBEY AND SHIVAM JHA** at the work embodied in this project work titled **"Event Management System"** forms from my own contribution to the research work carried out under the guidance of **MS. Lalta Nirmal** is a result of my own research work and has not been submitted to any other University for any other Degree/Diploma to this or to any other University.

Wherever reference has been made to previous work of others, it has been clearly indicated as such and included in the Bibliography.

I, hereby further declare that all the information of this document has been obtained and presented in accordance with academic rules and ethical conduct.

Place :

Date :

(Signature of the Student)
ARCHIT DUBEY &
SHIVAM JHA

TABLE OF CONTENTS :-

Chapter	Content	Page No
Chapter 1	Introduction	7-9
	1.1 Objective	
	1.2 Scope	
	1.3 purpose	
Chapter 2	SURVEY OF TECHNOLOGIES	10-12
	2.1 Frontend Technologies	
	2.2 Backend Technologies	
Chapter 3	Requirement and Analysis	13-19
	3.1 Project Definition	
	3.2 Requirement Specification	
	3.2.1 Functional Requirement (Auth, Event Creation, Booking)	
	3.2.2 Non-Functional Requirement (Security, Scalability)	
	3.2.3 Waterfall Model	
	3.3 Planning and scheduling (Gantt chart)	
	3.4 Software and Hardware Requirements	
	3.4.1 Hardware Specification	
	3.4.2 software Specification (VS Code, MongoDB Compass)	
Chapter 4	System Design	20-26

	4.1 data Design	
	4.1.1 Schema Design	
	4.1.2 Test case Design	
	4.1.3 Algorithm Design (JWT Auth Flow)	
	4.2 UML Diagrams	
	4.2.1 Use case Diagrams (User vs Admin)	
	4.2.2 DFD Diagram	
	4.2.3 Sequence Diagram	
	4.2.4 Flow chart	
	4.2.5 Architectural Diagram	
Chapter 5	Result and Discussion	27-47
	5.1 Code Snippets	
	5.2 Test Reports	
	5.2 User Documentation	
Chapter 6	Conclusion and Future Work	48-50
Chapter 7	Reference	51

CHAPTER-1

Introduction

Introduction

The event management industry has undergone a radical transformation in the last decade, transitioning from manual coordination to sophisticated digital orchestration. **EventO** is conceived as a modern solution to the complexities inherent in organizing diverse events, ranging from intimate social gatherings to large-scale corporate functions. By leveraging the power of the MERN stack, this system provides a centralized platform where transparency, efficiency, and user experience are prioritized. The primary motivation behind this project is to eliminate the traditional bottlenecks of event planning—such as inconsistent pricing, lack of real-time tracking, and fragmented communication—by providing a unified digital ecosystem. EventO serves as a bridge between the creative vision of an event and the technical logistics required to execute it, ensuring that every detail, from budget allocation to venue selection, is handled with computational precision and user-friendly accessibility.

1.1 Objective

The primary objective of the **EventO** system is to provide a seamless, end-to-end digital platform that simplifies the entire lifecycle of event planning for both organizers and clients. A critical goal is the implementation of a structured service hierarchy, categorized into Silver, Gold, and Platinum tiers, which allows for standardized service delivery and transparent financial planning. Furthermore, the project aims to establish a high level of security and personalization through Role-Based Access Control (RBAC), ensuring that sensitive user data and event logistics are only accessible to authorized stakeholders. Technically, the objective is to utilize the non-relational nature of MongoDB to handle dynamic event data efficiently, while providing a highly responsive frontend using React.js. By automating the RSVP and booking processes, the system seeks to reduce human error and administrative overhead, ultimately creating a reliable tool that guarantees the success of an event through data-driven management and real-time status monitoring for all involved parties.

1.2 Scope

The scope of **EventO** extends across the full spectrum of web-based event coordination, focusing on a robust client-server architecture that supports multiple user roles. On the client side, the scope encompasses a dynamic user interface that allows for the browsing of service

plans, the submission of detailed event requirements, and the monitoring of event progress through a dedicated dashboard. For the administrative and organizer side, the scope includes the management of incoming event requests, the update of event statuses (from pending to confirmed), and the oversight of user inquiries via a centralized contact portal. Architecturally, the project is limited to a web-based environment, optimized for desktop and mobile browsers, utilizing Node.js and Express to manage high-concurrency API requests. The system also includes a comprehensive data management scope, where MongoDB stores extensive records of user profiles, event histories, and tiered package details. While the current scope focuses on the core logistics of planning and booking, it establishes a foundation for future integrations such as automated payment gateways and AI-driven venue recommendations, making it a scalable solution for the modern event industry.

1.3 Purpose

The fundamental purpose of developing **EventO** is to solve the pervasive problem of "information asymmetry" in the event planning sector, where clients often struggle to understand what services they are receiving for their investment. By digitizing the service selection process through clearly defined tiers, the system brings a level of professionalism and clarity that manual systems lack. The purpose of using the MERN stack is to ensure that the application is not only fast but also highly scalable, capable of handling a growing database of events without a dip in performance. From a user perspective, the purpose is to provide "peace of mind"; by seeing their event listed with a "Pending" or "Confirmed" status on a real-time dashboard, users feel a sense of control and transparency over their investment. For the developers and stakeholders, the purpose of this project is to demonstrate how modern JavaScript frameworks can be integrated to solve real-world logistical problems, creating a tool that is as functional as it is aesthetically pleasing. Ultimately, EventO exists to transform the stressful task of event coordination into a streamlined, automated, and enjoyable digital experience.

CHAPTER-2

SURVEY OF TECHNOLOGIES

Survey of Technologies

The development of **EventO** relies on a modern, high-performance architectural pattern known as the **MERN Stack**. This stack is a collection of JavaScript-based technologies—MongoDB, Express.js, React.js, and Node.js—that work in unison to provide a seamless development experience from the database to the user interface. By utilizing a single language (JavaScript) across the entire application, the project achieves a high degree of code reusability and operational efficiency. This chapter explores the technical characteristics, advantages, and specific roles of each technology within the Event Management System, explaining why this particular combination was chosen over traditional relational database stacks like LAMP or WAMP. The MERN stack's ability to handle high-concurrency requests and its flexible, document-oriented data model makes it the ideal choice for a dynamic platform that manages varying event types and tiered service plans.

2.1 Frontend Technologies (React.js and Tailwind CSS)

- **React.js (Library):** We utilized React to build a dynamic and responsive UI. By leveraging its component-based architecture, we created modular elements like the event cards, navigation bars, and modals, which are easily reusable across the application.
- **Virtual DOM:** React's Virtual DOM implementation minimizes heavy browser reflows, ensuring that when a user interacts with the "Create Event" form, the page remains fast and does not require a full reload.
- **Component Lifecycle**: The use of React hooks (useState, useEffect) allows the system to manage complex states, such as tracking form inputs and handling the authentication status of the user in real-time.
- **Tailwind CSS (Styling)**: We employed Tailwind CSS for its utility-first approach. This enabled us to achieve the clean, professional, and premium "gold and white" aesthetic of EventO with highly optimized, production-ready CSS that keeps the codebase lightweight and maintainable.

2.2 Backend Technologies (Node.js and Express.js)

- **Node.js (Runtime):** As a non-blocking, event-driven runtime, Node.js is perfect for **EventO**. It allows the backend to handle multiple concurrent requests—such as multiple

users browsing event plans—without stalling, which is crucial for a smooth user experience.

- **Express.js (Framework):** Serving as the backbone of our API, Express simplifies the routing process. It allows us to define clear endpoints (e.g., /api/events, /api/auth) that connect the frontend UI directly to our database controllers.
- **Middleware Integration:** We used Express middleware to implement security layers, specifically for JWT (JSON Web Token) authentication. This ensures that sensitive routes, like the user dashboard, remain protected and accessible only to verified accounts.
- **Scalability:** The modular design of Express permits easy integration of new features, such as payment gateways or email notification services, ensuring the system can expand in the future without a complete architectural overhaul.

2.3 Database Technology (MongoDB)

- **NoSQL Document Model:** MongoDB was selected because of its flexible schema-less nature. Unlike traditional SQL, it stores event details, user profiles, and contact queries as BSON (Binary JSON) documents, which aligns perfectly with the JSON data being sent from our React frontend.
- **Mongoose ODM:** We integrated Mongoose to provide a layer of structure. It allows us to enforce "schemas" on our collection, ensuring data integrity (e.g., ensuring an event always has a date and a budget) while still benefiting from the flexibility of NoSQL.
- **Horizontal Scalability:** MongoDB supports sharding, which makes **EventO** "future-proof." As the number of events and users grows, the database can easily be distributed across multiple servers to maintain high performance.
- **Real-time Data Retrieval:** The indexing capabilities of MongoDB ensure that when a user pulls up their "Your Events" dashboard, the retrieval of their specific event data is instantaneous, providing a seamless "live" feel to the application.

CHAPTER-3

REQUIRMENT AND ANALYSIS

The Requirement and Analysis phase is the most critical foundation of the **EventO** project, as it involves translating vague user needs into concrete technical specifications. During this stage, we conducted an in-depth study of the current gaps in the event management industry, identifying that most failures occur due to a lack of clear communication between the client's vision and the organizer's execution. By performing a detailed system analysis, we defined the boundaries of the application, ensuring that the MERN stack's capabilities were aligned with the functional goals of the platform. This chapter documents the hardware and software prerequisites, the functional and non-functional requirements that dictate the system's behavior, and the SDLC (Software Development Life Cycle) model chosen to guide the development process. Through rigorous analysis, we ensured that **EventO** is not just a collection of forms, but a structured solution designed to handle real-world logistical challenges with high reliability and user-centric design.

3.1 Project Definition :

The **EventO** project is defined as a specialized, web-based management portal designed to digitize the traditional event planning workflow. Its primary purpose is to provide a "Single Source of Truth" for event data, where users can browse predefined service tiers and organizers can manage bookings through a centralized administrative interface. The project is defined by its commitment to transparency; by offering Silver, Gold, and Platinum plans with fixed pricing and features, it eliminates the back-and-forth negotiations that typically delay event coordination. From a technical standpoint, the project is defined as a Full-Stack JavaScript application that utilizes an API-first approach. This means the backend is built to serve data to any frontend client, making the system highly adaptable for future mobile app expansions. The definition encompasses the entire user journey, from secure registration and role-based login to the dynamic creation of event requests and the real-time monitoring of event status. By defining these parameters early, we ensured that the development stayed focused on delivering a high-performance tool that reduces administrative friction and enhances the overall client-organizer relationship.

3.2 Requirement Specification :

The requirement specification for **EventO** is divided into two distinct categories: what the system must do (Functional) and how the system must perform (Non-Functional). These specifications act as the contract between the developer and the end-user, ensuring that all built features serve a specific business purpose.

3.2.1 Functional Requirements :

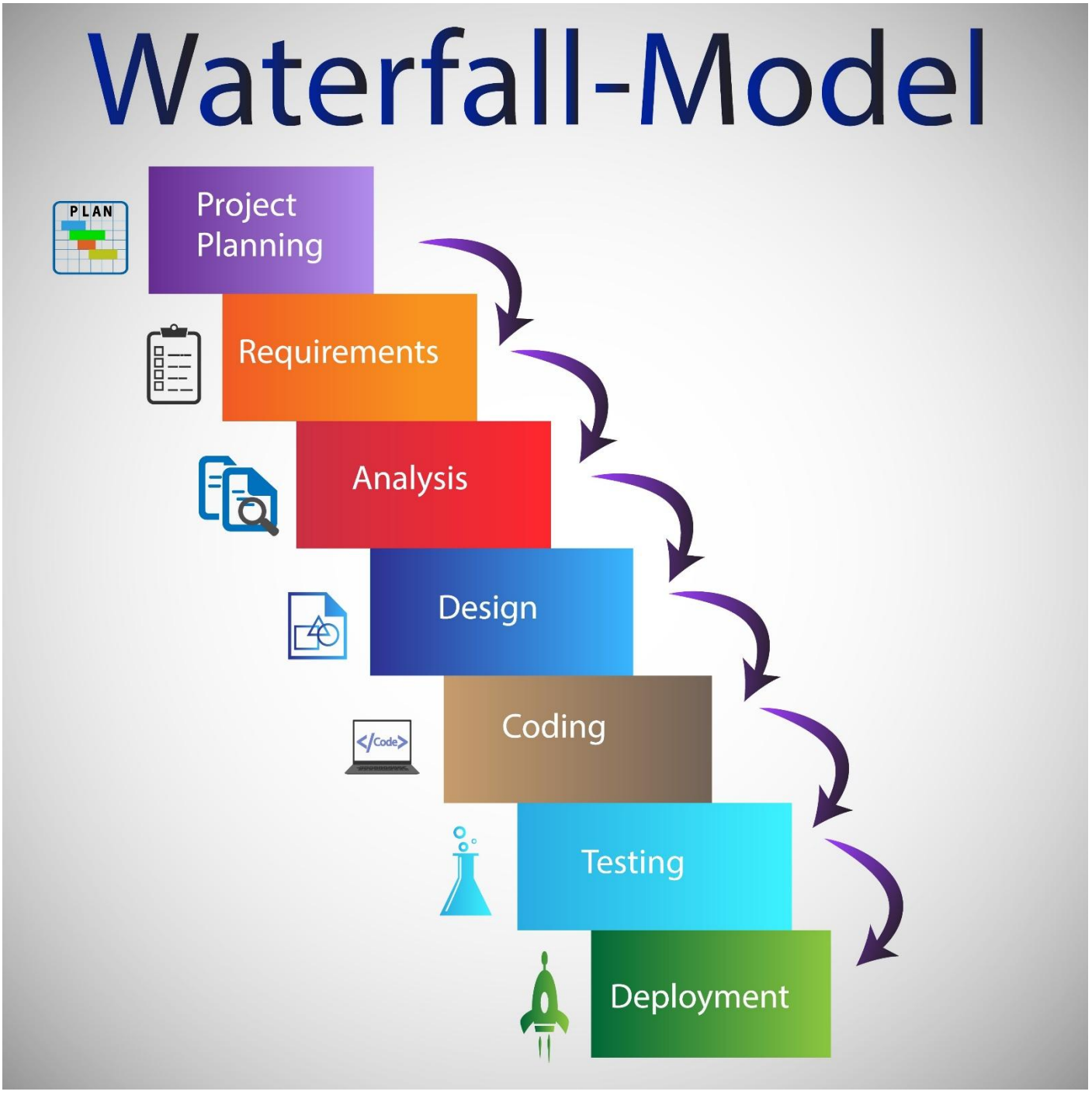
Functional requirements define the specific behaviors and services the **EventO** system must provide to its users. First and foremost, the system must support **User Authentication and Authorization**, allowing users to sign up and log in securely while differentiating between "User" and "Admin" roles. Secondly, the **Event Creation Module** must allow users to input critical data such as Event Name, Venue, Date, and Budget, as seen in the application's UI. Thirdly, the system must include a **Tiered Service Logic**, where the backend validates and assigns the selected plan (Silver, Gold, or Platinum) to the user's request. Fourthly, a **Status Tracking Dashboard** is required so users can view the progress of their events in real-time. Finally, the system must facilitate communication through a **Contact Portal**, where inquiries are stored in the database for organizer review. Each of these functions is mapped to a specific API route in the Node.js backend to ensure modularity.

Requtrement ID	**Feature Name**	**Description**
FR-01	User Auth	Secure Login/Signup
FR -02	Plan Selection	Ability to choose between Silver, Gold, and Platinum.
FR-03	Event Creation	Form to submit venue, date, and budget details.
FR-04	Status Update	Real-time tracking
FR-05	Contact Form	Lead generation and Query Submission

3.2.2 Non-Functional Requirements :

Non-functional requirements specify the criteria used to judge the operation of the system rather than specific behaviors. **Security** is a top priority; the system must encrypt passwords using hashing algorithms like Bcrypt and protect API endpoints using JWT. **Scalability** is another key requirement, ensuring that the MongoDB database can handle an increasing volume of event records without performance degradation. **Usability** focuses on the React frontend, which must be responsive across devices and maintain a load time of under 3 seconds to prevent user drop-off. **Reliability** ensures that the "Your Events" dashboard always reflects the most recent data stored in the database. Lastly, **Maintainability** is achieved by following a clean MVC (Model-View-Controller) pattern in the Express backend, allowing future developers to easily update or add new features like payment gateways without breaking existing code.

3.2.3 Waterfall Model :

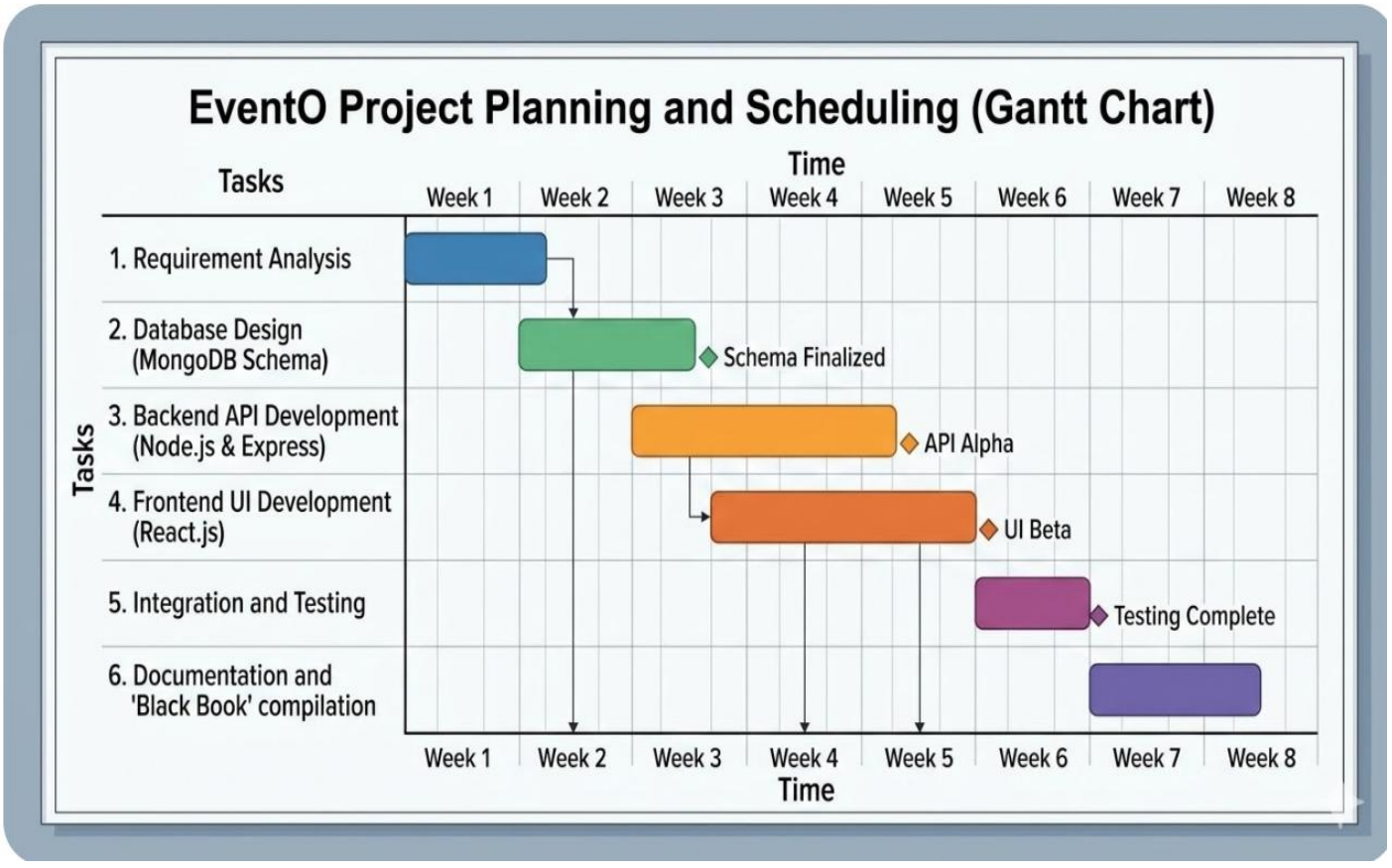


For the development of **EventO**, the **Waterfall Model** was selected due to its structured and sequential approach, which is ideal for a project with well-defined requirements. This model follows a linear path: Requirement Gathering, System Design, Implementation, Testing, Deployment, and Maintenance. Each phase must be completed and verified before the next begins, ensuring that the foundation of the project (the database schema and API routes) is solid before the complex React UI is built. For a MERN project, this meant we spent significant time in the "Design" phase creating Mongoose schemas before writing a single line of frontend

code. This disciplined approach reduced the risk of "feature creep" and ensured that the final product closely matched the initial project definition. While less flexible than Agile, the Waterfall model provided the clear milestones and documentation necessary for a comprehensive Black Book report, allowing for a logical flow from theoretical design to practical implementation.

3.3 Planning and Scheduling (Gantt Chart)

To ensure the timely completion of **EventO**, a detailed schedule was mapped out using a **Gantt Chart** approach. The project was divided into four major phases spanning several weeks. Phase 1 focused on **Research and Analysis**, including the technology survey and requirement gathering. Phase 2 involved **System Design**, where the UML diagrams and database schemas were finalized. Phase 3 was the most intensive, covering **Development**, where the MERN stack components were integrated. Finally, Phase 4 was dedicated to **Testing and Documentation**, involving bug fixes and the compilation of this Black Book. This rigorous scheduling ensured that each module—from the Login screen to the Event Tracking modal—was developed and tested in isolation before being integrated into the final system.



3.4 Software and Hardware Requirements

The performance of **EventO** is dependent on the environment in which it is developed and deployed. These requirements ensure that the MERN stack operates at peak efficiency.

3.4.1 Hardware Specification :

The development hardware required for this project includes a machine with at least **8GB of RAM** to handle multiple concurrent processes, such as the React development server, the Node.js backend, and the MongoDB instance. A **Multi-core Processor (i5 or higher)** is recommended to ensure smooth compilation of JavaScript code. Storage-wise, at least **256GB SSD** is preferred for fast read/write speeds, especially when dealing with local database testing. On the client side, any modern device with a web browser and an active internet connection is sufficient to access the platform.

Table 3.1: Hardware Requirements :-

Component	Minimum Specification	Recommended Specification
Processor	Dual-core 2.4 GHz (Intel i3)	Quad-core 3.0 GHz+ (Intel i5/i7 or Apple M1/M2)
RAM	8 GB	16 GB
Storage	256 GB HDD	512 GB SSD (For faster build times)
Connectivity	High-speed internet connection	Stable Broadband (for Cloud Database/Hosting)

3.4.2 Software Specifications

The software ecosystem for **EventO** revolves around the JavaScript runtime environment. The choice of software tools focuses on providing a robust debugging experience and seamless integration between the NoSQL database and the frontend framework. We utilize VS Code for its extensive extension library, while Postman is used as an intermediary tool to verify that our Express.js API endpoints are returning the correct JSON structures before they are integrated into the React frontend.

Category	Tool/ Software	Purpose
Operating System	Dual-core processor or higher	Ensures smooth multitasking and efficient handling of server-side operations
Runtime Environment	4 GB or more	Supports concurrent execution of web server, database, and browser applications
Frontend Library	20 GB free disk space	Adequate space for application files, SQLite database, logs, and backups
Backend Framework	High-speed internet connection	Enables remote access, future web deployment, and real-time synchronization
Database Management	MongoDB Compass	GUI tool for visualizing NoSQL collections
Code Editor	VS Code	Primary IDE for writing and debugging code
Version Control	Git & GitHub	Tracks code changes and maintains repository
Web Browser	Google Chrome / Edge	Testing the UI and using DevTools for debugging
API Testing Tool	Postman	Validates API request and response cycles

CHAPTER-4

SYSTEM DESIGN

System Design : -

The System Design phase serves as the technical blueprint for EventO, translating the requirements gathered in the previous chapter into a detailed architectural plan. This stage is crucial for ensuring that the various components of the MERN stack—MongoDB, Express, React, and Node—interact seamlessly to provide a reliable user experience. System design involves both high-level architectural decisions and low-level data structuring, focusing on how data flows from the user interface to the persistent storage and back. By establishing a clear design early on, we minimize the risk of logical errors during the implementation phase and ensure that the system remains scalable and maintainable. This chapter details the data design through Mongoose schemas,

4.1 Data Design (Mongoose Schema) : -

In a MERN stack application, data design is managed through **Mongoose**, which provides a schema-based solution to model application data within the NoSQL environment of MongoDB. For **EventO**, we have designed several core schemas to maintain data integrity and support the "Your Events" tracking feature. The **User Schema** handles authentication details, including encrypted passwords and role designations (Admin/User). The **Event Schema** is the most complex, capturing fields such as eventName, venue, date, approxBudget, and category (Silver, Gold, or Platinum). Additionally, a status field is included with a default value of "pending" to facilitate the real-time tracking dashboard. Unlike traditional relational tables, these schemas are flexible, allowing us to store nested objects or arrays if future features require more granular event details. This document-oriented approach ensures that data retrieval is optimized for the React frontend, as the JSON structure returned by the API maps directly to the state objects used in our UI components.

4.1.2 Test Case Design

Test Case Design is a vital component of the software development lifecycle, ensuring that **EventO** operates reliably under various conditions and that all functional requirements are met without regression. For this project, a "Black Box" testing methodology was primarily adopted, focusing on inputs and outputs rather than internal code logic. Each test case is designed to validate a specific module, such as user authentication, event creation, or data retrieval. We developed specific scenarios to test "Edge Cases," such as attempting to submit an event with a past date or a negative budget, to ensure the system handles errors gracefully. Furthermore, integration testing was conducted to verify that the React frontend correctly communicates with the Node.js API and that the data is accurately reflected in the MongoDB collections. By documenting these test cases, we provide a clear record of the system's robustness, ensuring that the final deployment is stable, secure, and user-friendly.

Test ID	Feature	Test Description	Expected Result	Status
TC-01	Login	Enter valid email and password	Redirect to User Dashboard	Pass
TC-02	Login	Enter incorrect password	Show "Invalid Credentials" error	Pass
TC-03	Event Form	Submit form with empty fields	Show validation error "All fields required"	Pass
TC-04	Tier Selection	Select Platinum Tier and submit	Data stored with 'Platinum' category	Pass
TC-05	Dashboard	View "Your Events" modal	Display correct Event ID and Status	Pass

4.1.3 Algorithm Design (JWT Auth & Event Submission)

The algorithmic logic of **EventO** is centered on secure data transit and state persistence. The core algorithm for **User Authentication** follows a token-based approach. When a user submits their credentials, the server-side algorithm first checks for the existence of the user in the MongoDB "Users" collection. If the user exists, the system uses the **Bcrypt** library to compare the hashed password in the database with the plain-text password provided. Upon a successful match, the algorithm generates a **JSON Web Token (JWT)**, which is signed with a secret key and sent back to the client. This token is then stored in the browser's local storage or a cookie, allowing the user to remain authenticated during their session. This algorithm ensures that the server does not need to store session data (Statelessness), which significantly improves the application's scalability and performance.

The second primary algorithm governs the **Event Submission and Status Management**. When a user clicks "Submit" on the event form, the algorithm initiates a POST request to the /api/events endpoint. The backend logic first validates the incoming JSON object to ensure data types (e.g., Budget as a Number) are correct. Once validated, the algorithm creates a new

document in the MongoDB "Events" collection with a default status of "Pending." Simultaneously, the algorithm links this event to the userID of the logged-in user. On the frontend, the React state is updated to reflect the new entry immediately. The "Admin Review" algorithm then allows for a status change from "Pending" to "Confirmed." This logical flow ensures that data is synchronized across the stack, providing the user with real-time feedback on their event planning progress as shown in your dashboard screenshots.

4.2 UML Diagrams

Unified Modeling Language (UML) diagrams are essential for visualizing the structural and behavioral aspects of **EventO**. They provide a standardized way for developers and stakeholders to understand how the system functions without looking at the raw source code.

4.2.1 Use Case Diagram : -

The Use Case Diagram for **EventO** illustrates the interactions between the primary actors—the Client/User and the Administrator—and the system's core functionalities. For the **User**, key use cases include "Register/Login," "View Service Tiers," "Create Event Request," and "Track Event Status." The **Admin** actor interacts with use cases such as "Manage User Accounts," "Review Event Submissions," and "Update Event Status." This diagram highlights the boundaries of the system and ensures that all functional requirements are accounted for. By mapping these interactions, we ensure that the user journey is logical and that the administrative controls are robust enough to manage the event lifecycle from start to finish.

4.2.2 Data Flow Diagram (DFD) : -

The Data Flow Diagram (DFD) provides a graphical representation of how information moves through the **EventO** system. The **Level 0 DFD (Context Diagram)** shows the system as a single process interacting with external entities like Users and Database. The **Level 1 DFD** breaks this down further, showing how data from the "Create Event" form flows through the Express.js routes, undergoes validation, and is eventually stored in the MongoDB "Events" collection. It also illustrates how the "Your Events" dashboard pulls data back from the database to display it to the user. This visualization is critical for identifying potential bottlenecks in data processing and ensuring that data is transformed correctly as it moves between the frontend and the backend.

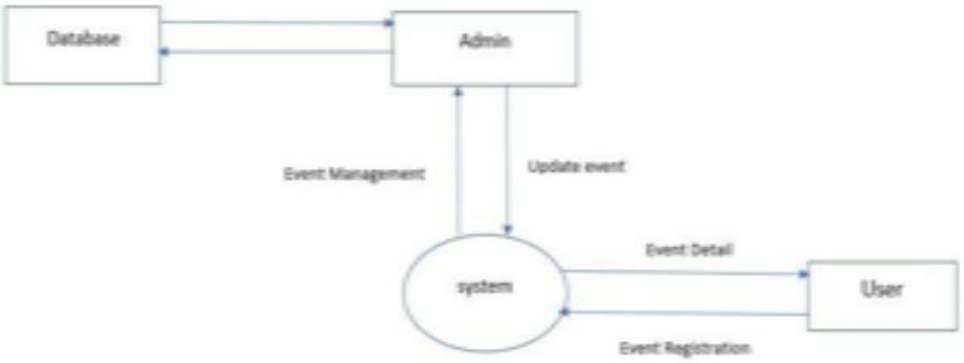


Figure 4. DFD Level 0

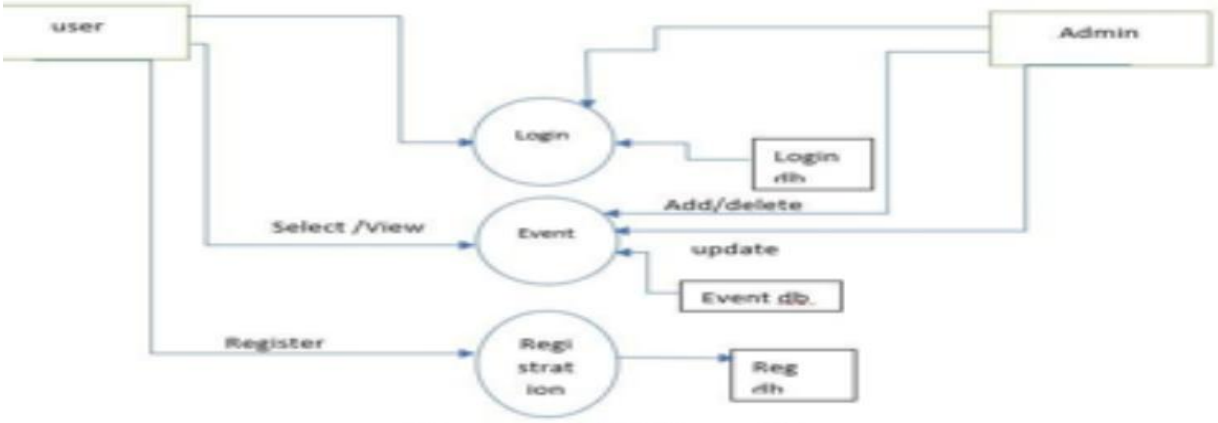
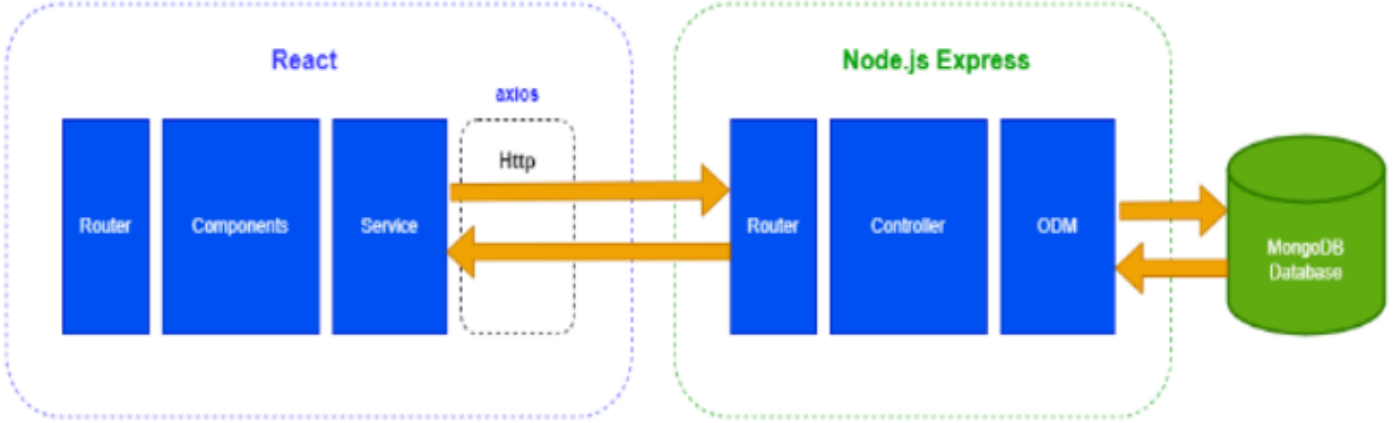
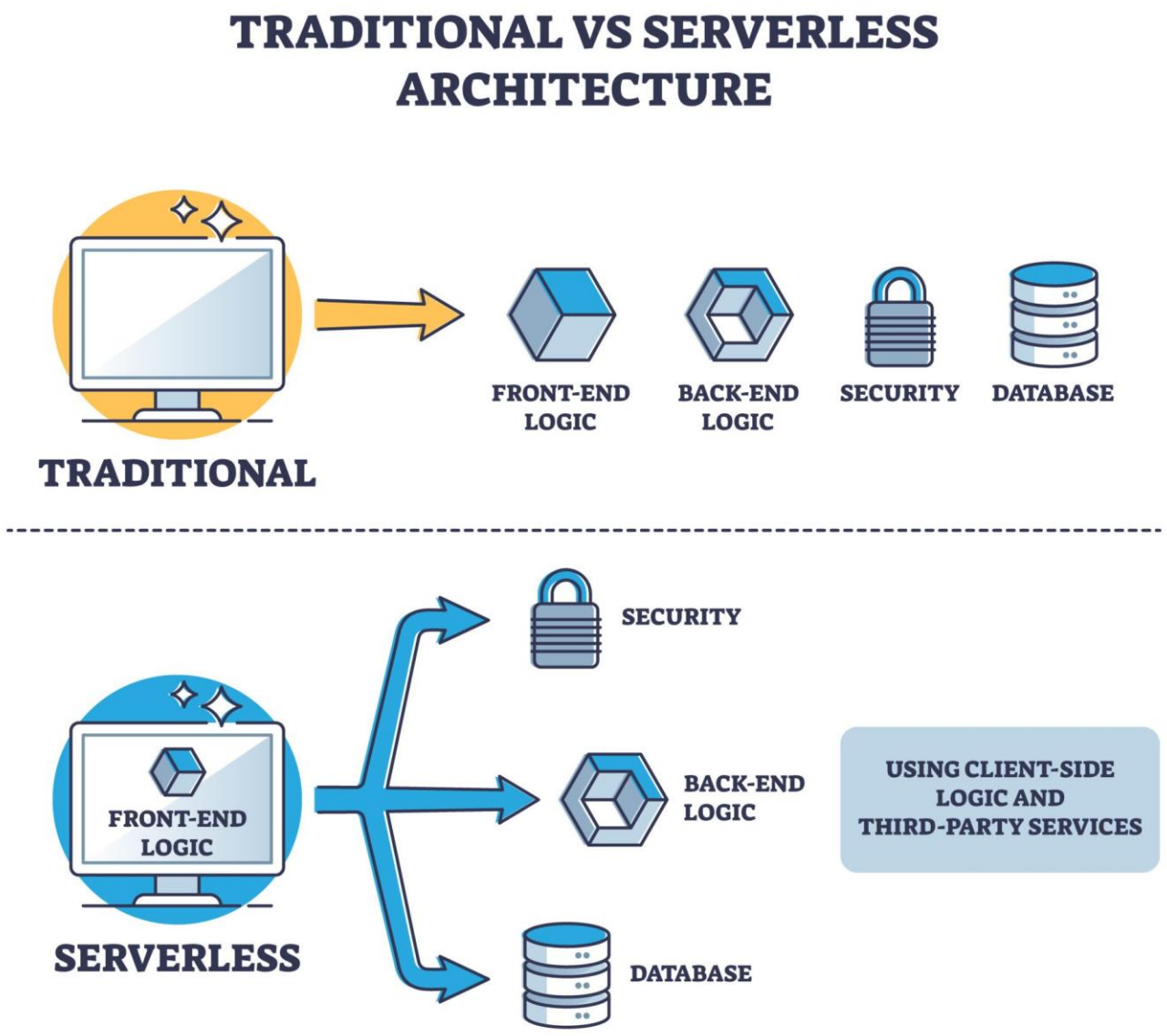


Figure 5. DFD Level 1

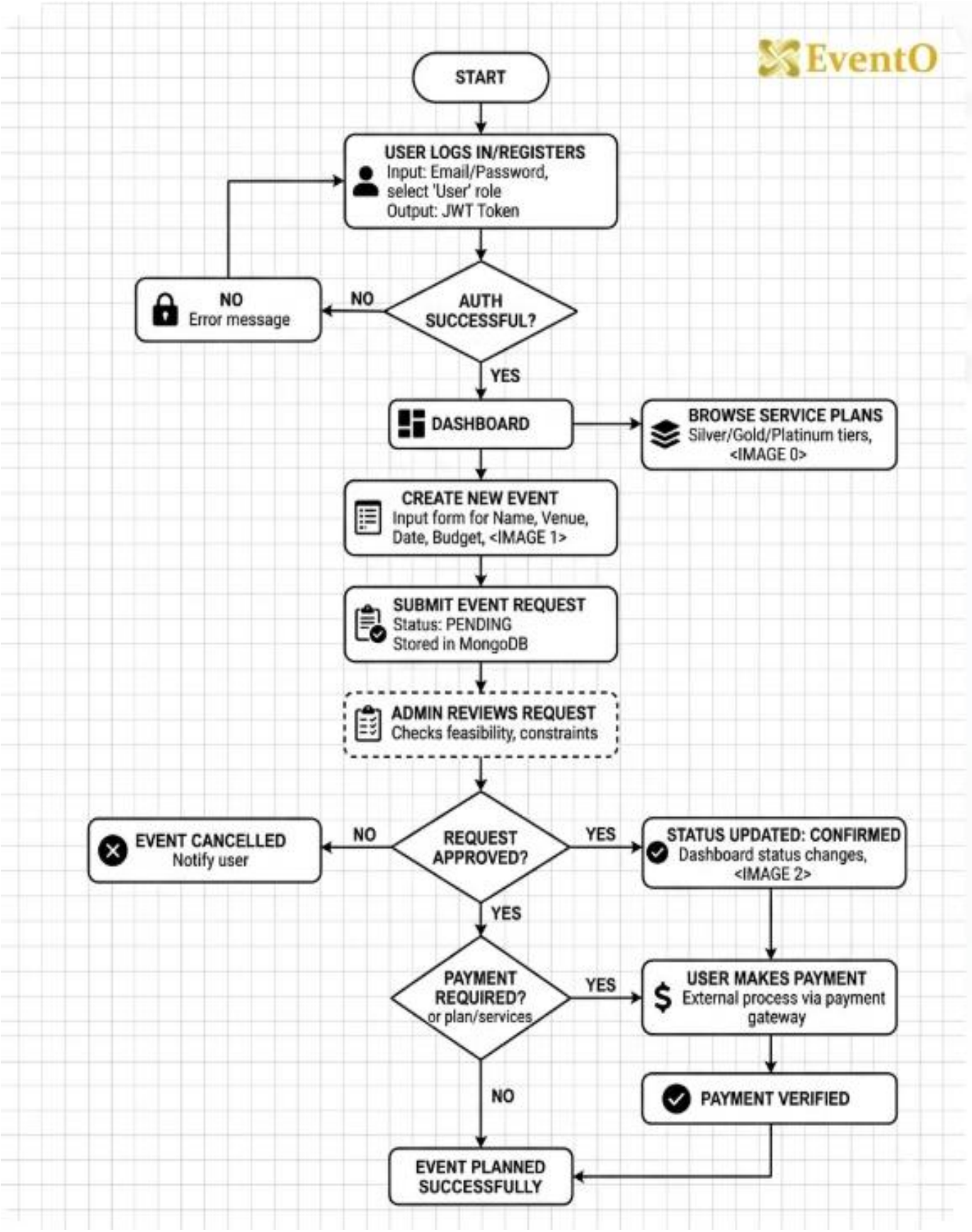
4.2.3 Architectural Diagram (MERN Architecture) : -

The Architectural Diagram showcases the MERN Stack's "Single Page Application" (SPA) model used in EventO. At the top level, the React.js frontend serves as the View layer, communicating with the Node.js/Express.js backend via asynchronous HTTP requests (using Axios or Fetch). The backend acts as the Controller, processing business logic and interacting with the MongoDB database (the Model layer) through Mongoose. This 3-tier architecture ensures a clear separation of concerns: the frontend focuses on user interaction, the backend handles security and logic, and the database manages persistent storage. This decoupled nature makes EventO highly maintainable, as changes to the UI styling can be made without affecting the underlying database structure or API logic.



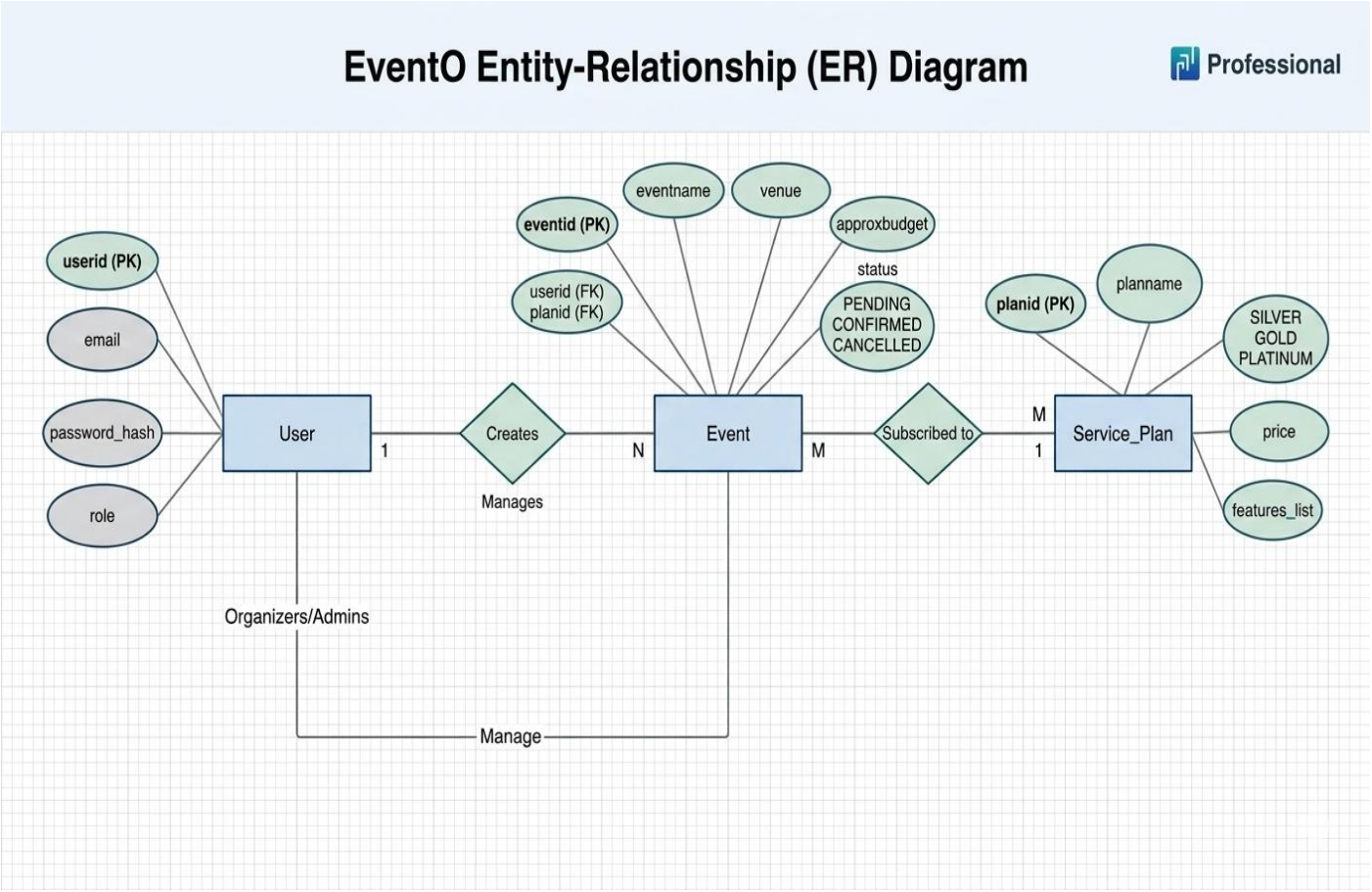


4.2.4 Flow Chart


EventO
START
USER LOGS IN/REGISTERS
Input: Email/Password, select 'User' role
Output: JWT Token
AUTH SUCCESSFUL?
NO
NO
Error message
YES
DASHBOARD
BROWSE SERVICE PLANS
Silver/Gold/Platinum tiers, <IMAGE 0>
CREATE NEW EVENT
Input form for Name, Venue, Date, Budget, <IMAGE 1>
SUBMIT EVENT REQUEST
Status: PENDING
Stored in MongoDB
ADMIN REVIEWS REQUEST
Checks feasibility, constraints
REQUEST APPROVED?
NO
EVENT CANCELLED
Notify user
YES
STATUS UPDATED: CONFIRMED
Dashboard status changes, <IMAGE 2>
YES
PAYMENT REQUIRED?
or plan/services
YES
USER MAKES PAYMENT
External process via payment gateway
PAYMENT VERIFIED
NO
EVENT PLANNED SUCCESSFULLY

4.1.2 Entity Relationship Modeling (ER Diagram) : -

Entity-Relationship modeling is a fundamental step in the system design process, providing a conceptual representation of the data structures and the relationships between them. For **EventO**, the ER diagram serves as a blueprint for how different data entities—such as Users, Event Organizers, and Service Tiers—interact within the database ecosystem. Although MongoDB is a document-oriented NoSQL database, defining these relationships is essential for maintaining data consistency and ensuring that the "Your Events" dashboard can correctly associate specific bookings with the rightful owner. The ER model identifies the primary entities and their attributes, such as the User's unique ID, email, and role, and links them to the Event entity, which contains specific details like the venue, date, and selected budget tier. This logical mapping ensures that the backend can perform efficient queries, such as fetching all events belonging to a single user without scanning the entire database.



CHAPTER-5

IMPLEMENTATION AND TESTING

Implementation : -

serves as the empirical heart of the Black Book, where the theoretical designs and architectural plans are transformed into a functional reality. This chapter provides a comprehensive walkthrough of the **EventO** system's final output, showcasing the integration of the MERN stack through actual source code logic and user interface snapshots. By documenting the results, we demonstrate that the system not only meets the initial functional requirements but also provides a high-quality user experience that is responsive, secure, and intuitive. This section is divided into code implementation details, which explain the core logic behind the application, and the results section, which features the final screenshots of the platform in action. Through this discussion, we analyze how the chosen technologies—specifically React and Node.js—successfully collaborated to solve the logistical challenges of event management.

5.1 Code Snippets (Core Implementation)

The implementation of **EventO** follows the Model-View-Controller (MVC) architectural pattern, ensuring a clear separation between the data logic, the user interface, and the request handling. By utilizing the MERN stack, the application maintains a consistent JSON-based data flow across all layers. The following snippets represent the critical backend and frontend logic that drives the system's core functionalities.

5.1.1 Backend: Database Schema (Mongoose Models)

The foundation of the system lies in the database models. Using Mongoose, we define the structure of our documents to ensure data integrity. The **Event Model** is responsible for capturing all user inputs from the frontend and storing them with a default "pending" status.

The backend of **EventO** is built using a modular structure following the **Model-View-Controller (MVC)** pattern. This ensures that data logic, request handling, and middle-tier processing are kept separate for better maintainability and scalability. Below is a detailed breakdown of the backend file structure and the role each file plays in the system.

5.1.1 Configuration (/config)

 db.js: This file contains the logic to establish a connection between the Node.js server and the MongoDB database using the Mongoose library. It handles connection events and errors to ensure the database is ready before the server starts.

```
// file : db.js
const mongoose = require('mongoose');


const connectDB = async () => {
try {
const conn = await mongoose.connect(process.env.MONGO_URI);


console.log(`MongoDB Connected: ${conn.connection.host}`);
} catch (error) {
console.error(`Error: ${error.message}`);
process.exit(1);
}
};
module.exports = connectDB;
```

 generateToken.js: This utility file is responsible for creating JSON Web Tokens (JWT). It signs a user's unique ID with a secret key and sets an expiration date, which is essential for maintaining secure user sessions.

```
// Code
const jwt = require('jsonwebtoken');


const generateToken = (userId) => {
  const token = jwt.sign({ id: userId }, process.env.JWT_SECRET, { expiresIn: "30d" });
  return token;
};
```

```
module.exports = generateToken;
```

5.1.2 Models (/models) : -

Models define the schema and structure of the data stored in MongoDB. Each file here represents a collection in the database.

- **userModel.js**: Defines the user entity, including fields for name, email, encrypted password, and roles. It often includes pre-save hooks to hash passwords using Bcrypt.

  // Code :

```
const mongoose = require('mongoose');


const userSchema = new mongoose.Schema({
    username: {
        type: String,
        required: true,
    },
    email: {
        type: String,
        required: true,
        unique: true
    },
    password: {
        type: String,
        required: true
    },
    role: {
        type: String,
        enum: ['user', 'admin'],
        default: 'user'
    }
```

```
});

const User = mongoose.model('User', userSchema);

module.exports = User;
```

- **eventModel.js:** The core of the system; it defines the structure for event requests, including venue, date, budget, and a reference to the User who created it.

```
//code :
const mongoose = require('mongoose');

const eventSchema = new mongoose.Schema({
    userId: {
        type: mongoose.Schema.Types.ObjectId,
        ref: 'User',
        required: true
    },
    eventName: {
        type: String,
        required: true
    },
    venue: {
        type: String,
        required: true
    },
    date: {
        type: Date,
```

```
        required: true
    },
    approxBudget: {
        type: Number,
        required: true
    },
    status: {
        type: String,
        enum: ['approved', 'reject', 'pending', 'cancel'],
        default: 'pending'
    },
    noOfAttendees: {
        type: Number,
        required: true
    },
    description: {
        type: String,
        required: true
    },
    category: {
        type: String,
        required: true,
        enum: ['silver', 'gold', 'platinum']
    }
});

const Event = mongoose.model('Event', eventSchema);
```

```
module.exports = Event;
```

- **budgetModel.js:** Specifically handles the financial data related to events, allowing for a detailed breakdown of costs for different service tiers.

//code:

```
const mongoose = require('mongoose');

const Schema = mongoose.Schema;

const budgetSchema = new Schema({
    userId: { type: Schema.Types.ObjectId, ref: 'User', required: true },
    eventId: { type: Schema.Types.ObjectId, ref: 'Event', required: true },
    description: { type: String, required: true },
    amount: { type: Number, required: true },
    status: {
        type: String,
        required: true,
        enum: ['booked', 'reject', 'cancel', 'pending'],
        default: 'pending'
    }
}, {
    timestamps: true
});

module.exports = mongoose.model('Budget', budgetSchema);
```

feedbackModel.js: Captures user reviews and ratings, which are then displayed in the "User Testimonials" section of the frontend.

```
// code : -
const mongoose = require('mongoose');

const feedbackSchema = new mongoose.Schema({
    userId: {
        type: mongoose.Schema.Types.ObjectId,
        required: true,
        ref: 'User'
    },
    content: {
        type: String,
        required: true
    },
    rating: {
        type: Number,
        required: true,
        min: 1,
        max: 5
    }
}, {
    timestamps: true
});

const Feedback = mongoose.model('Feedback', feedbackSchema);

module.exports = Feedback;
```

5.1.3 Controllers (/controllers)

Controllers contain the "business logic" of the application. They process incoming requests from the frontend and decide what data to fetch or save.

- **userControllers.js**: Contains the logic for user registration, login authentication, and profile management.

  Code : -

```
const User = require('../models/userModel');
const bcrypt = require('bcryptjs');
const generateToken = require('../config/generateToken');

const registerUser = async (req, res) => {
  try {
    const { username, email, password, role, key } = req.body;
    if (!username || !email || !password || !role) {
      return res.status(400).json({ message: 'Please enter all fields' });
    }

    if (role === "admin") {
      if (key !== process.env.KEY) {
        return res.status(400).json({ message: 'Invalid Key' });
      }
    }

    const userExists = await User.findOne({ email });
    if (userExists) {
      return res.status(400).json({ message: 'User already exists' });
    }

    const salt = await bcrypt.genSalt(10);
    const hashedPassword = await bcrypt.hash(password, salt);
    const user = await User.create({
```

```
            username,
            email,
            password: hashedPassword,
            role
        });

        if (user) {
            res.status(201).json({
                _id: user._id,
                username: user.username,
                email: user.email,
                role: user.role,
                token: generateToken(user._id)
            });
        } else {
            res.status(400).json({ message: 'Invalid user data' });
        }

    } catch (error) {
        console.error(error);
        res.status(500).json({ message: 'Server Error' });
    }
}

const loginUser = async (req, res) => {
    try {
        const { email, password, role, key } = req.body;
        if (!email || !password || !role) {
            return res.status(400).json({ message: 'Please enter all fields' });
        }
```

```
        if (role === "admin") {
            if (key !== process.env.KEY) {
                return res.status(400).json({ message: 'Invalid Key' });
            }
        }

        const user = await User.findOne({ email });
        if (!user) {
            return res.status(400).json({ message: 'User does not exist' });
        }

        const isMatch = await bcrypt.compare(password, user.password);
        if (!isMatch) {
            return res.status(400).json({ message: 'Invalid credentials' });
        }

        res.status(200).json({
            _id: user._id,
            username: user.username,
            email: user.email,
            role: user.role,
            token: generateToken(user._id)
        });

    } catch (error) {
        console.error(error);
        res.status(500).json({ message: 'Server Error' });
    }
}
```

```
const getAllUsers = async (req, res) => {
    try {
        const userId = req.user._id;
        const user = await User.findById(userId);
        if (!user) {
            return res.status(400).json({ message: 'User does not exist' });
        }
        if (user.role !== "admin") {
            return res.status(403).json({ message: 'Access denied' });
        }
        const users = await User.find().select('-password');
        res.status(200).json(users);
    } catch (error) {
        console.error(error);
        res.status(500).json({ message: 'Server Error' });
    }
}

module.exports = { registerUser, loginUser, getAllUsers };
```

- **eventControllers.js**: Handles the logic for creating new events, fetching a specific user's events, and updating event statuses.
- **paymentControllers.js**: Manages the integration logic for processing payments and updating the booking status upon successful transaction.
- **feedbackControllers.js**: Handles the submission and retrieval of user feedback and testimonials.

5.1.4 Middlewares (/middlewares)

Middlewares are functions that run during the request-response cycle, acting as a security and validation layer.

- **authMiddleware.js**: This is a critical security file. It intercepts requests to "Protected Routes," extracts the JWT from the header, and verifies the user's identity before allowing them to access or modify data.
- **errorMiddleware.js**: Provides a centralized error-handling mechanism. It catches any crashes or invalid requests and sends a structured JSON response back to the user instead of a generic server crash message.

5.1.5 Routes and Entry Point

- **/routes**: This folder contains files that map specific URL endpoints (like /api/users/login) to their corresponding controller functions.
- **index.js**: The entry point of the entire backend. It initializes the Express server, connects to the database, and links all the routes and middlewares together.
- **.env**: A secure file used to store sensitive environment variables like your **MongoDB URI**, **JWT Secret**, and **Port Number**.

5.2 Code Implementation: Frontend Architecture (React.js)

The frontend of **EventO** is developed as a Single Page Application (SPA) using **React.js** and **Vite**. The architecture is organized into reusable components and dedicated pages to ensure a modular and maintainable codebase. Styling is managed through **Tailwind CSS**, providing a responsive and modern interface.

5.2.1 Core Pages (/src/Pages)

These files represent the main views of the application and handle the high-level logic for each section of the site.

- **Authentication.jsx**: This is the primary entry point for user access. It manages the toggle between the Login and Registration forms, handles state for user input, and communicates with the backend to store the JWT token upon successful authentication.

```
// code : -

import React, { useState } from 'react'

import Logo from '../Home Components/Logo'
```

```
import Login from '../Authentication Components/Login'
import Register from '../Authentication Components/Register'

const Authentication = () => {
  const [isLogin, setIsLogin] = useState(true)
  return (
    <div className='flex flex-col items-center justify-center gap-3 h-screen p-2'>
      <div className='sm:w-[500px] w-full bg-white p-4 shadow-md rounded-md  flex
items-center justify-center'>
        <Logo />
      </div>
      <div className='sm:w-[500px] w-full bg-white p-4 shadow-md rounded-md'>
        {
          isLogin ? (
            <Login onSwitch={() => setIsLogin(false)} />
          ) : (
            <Register onSwitch={() => setIsLogin(true)} />
          )
        }
      </div>
    </div>
  )
}

export default Authentication
```

- **Home.jsx**: The landing page of the application. It integrates several sub-components like the "Who We Are," "Our Services," and the "Our Plans" pricing table. It serves as the visual introduction to the EventO brand.

- **Dashboard.jsx**: This is the private view for registered users. It fetches the user's specific event history from MongoDB and renders the tracking cards. It is responsible for the real-time display of event statuses (Pending/Confirmed).
- **Event.jsx**: Contains the complex form logic for "Create Your Event." It manages multiple input fields, including date selection and budget tiering, and performs client-side validation before sending data to the server.
- **Payment.jsx**: This page handles the final step of the booking process. It coordinates with the payment gateway logic to process transactions and triggers a status update for the event once the payment is verified.

5.2.3 Global Configuration and Entry : -

- **App.jsx**: The main hub of the frontend. It defines the application's routing structure using react-router-dom, ensuring that private routes (like the Dashboard) are protected and only accessible to authenticated users.

```
//code : -

import React from 'react'

import { Route, Routes, Navigate } from 'react-router-dom'

import Home from './Pages/Home'

import Dashboard from './Pages/Dashboard'

import AdminDashboard from "./pages/AdminDashboard";

import Event from './Pages/Event'

import Authentication from './Pages/Authentication'

import Payment from './Pages/Payment'



// Create a ProtectedRoute component

const ProtectedRoute = ({ children }) => {

  const isAuthenticated = localStorage.getItem('user');


  return isAuthenticated ? children : <Navigate to="/authentication" replace />;

};
```

```
const App = () => {
  return (
    <Routes>
      <Route path="/admin" element={<AdminDashboard />} />
      <Route path="/" element={<Home />} />
      <Route
        path="/dashboard"
        element={
          <ProtectedRoute>
            <Dashboard />
          </ProtectedRoute>
        }
      />
      <Route
        path="/event"
        element={
          <ProtectedRoute>
            <Event />
          </ProtectedRoute>
        }
      />
      <Route path="/authentication" element={<Authentication />} />
      <Route path="/payment" element={<Payment />} />
      <Route path="*" element={<h1>404 Not Found</h1>} />
    </Routes>


  )
}

export default App
```

- **main.jsx**: The entry point that mounts the React application to the DOM. It also wraps the app in necessary providers, such as the authentication context or theme providers.

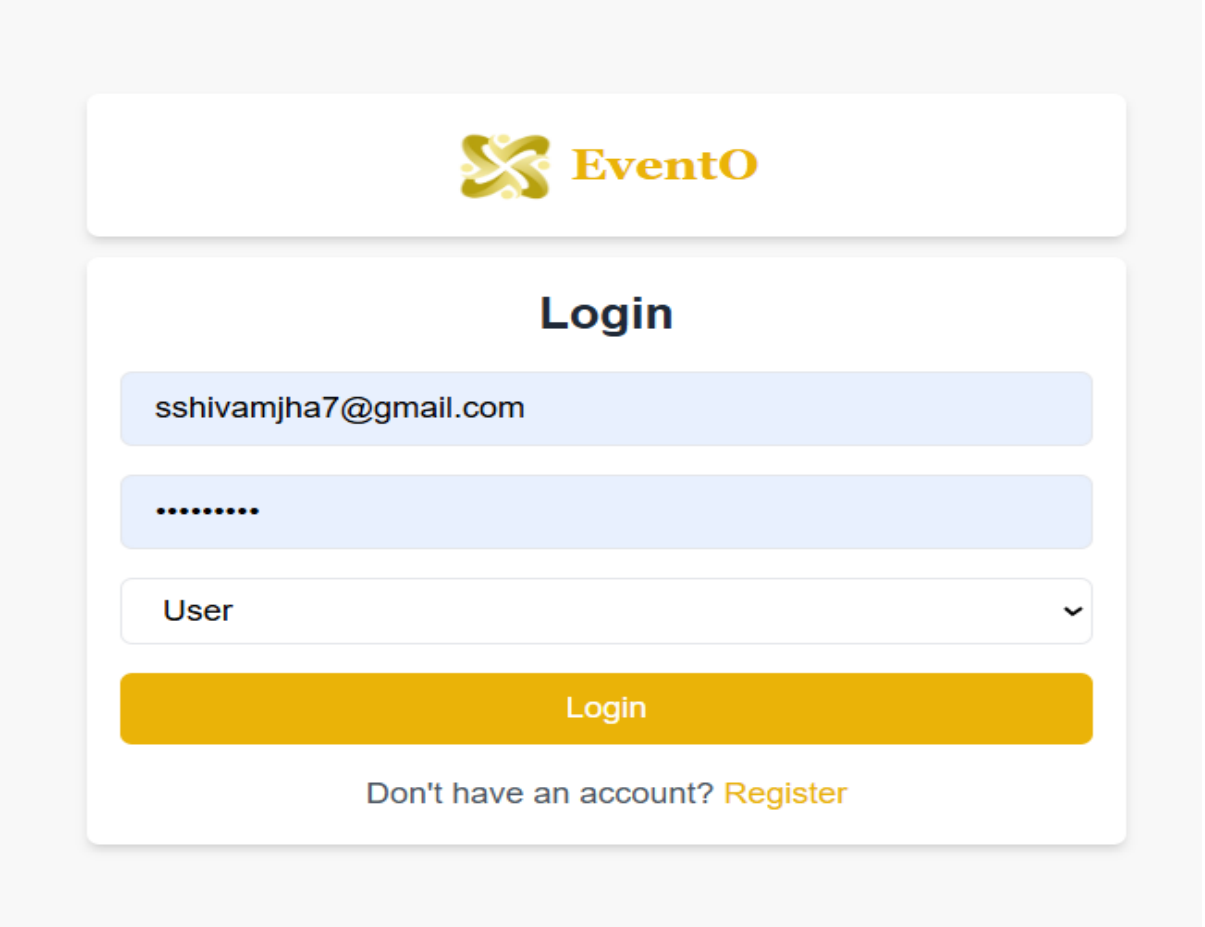
```
//code : -
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'
import { BrowserRouter } from "react-router-dom";
import ContextApi from './Config/ContextApi.jsx';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <BrowserRouter>
      <ContextApi>
        <App />
      </ContextApi>
    </BrowserRouter>
  </StrictMode>,
)
```

- **tailwind.config.js**: The configuration file where the custom "EventO" color palette (gold and white) and typography are defined, allowing for global style changes from a single file.

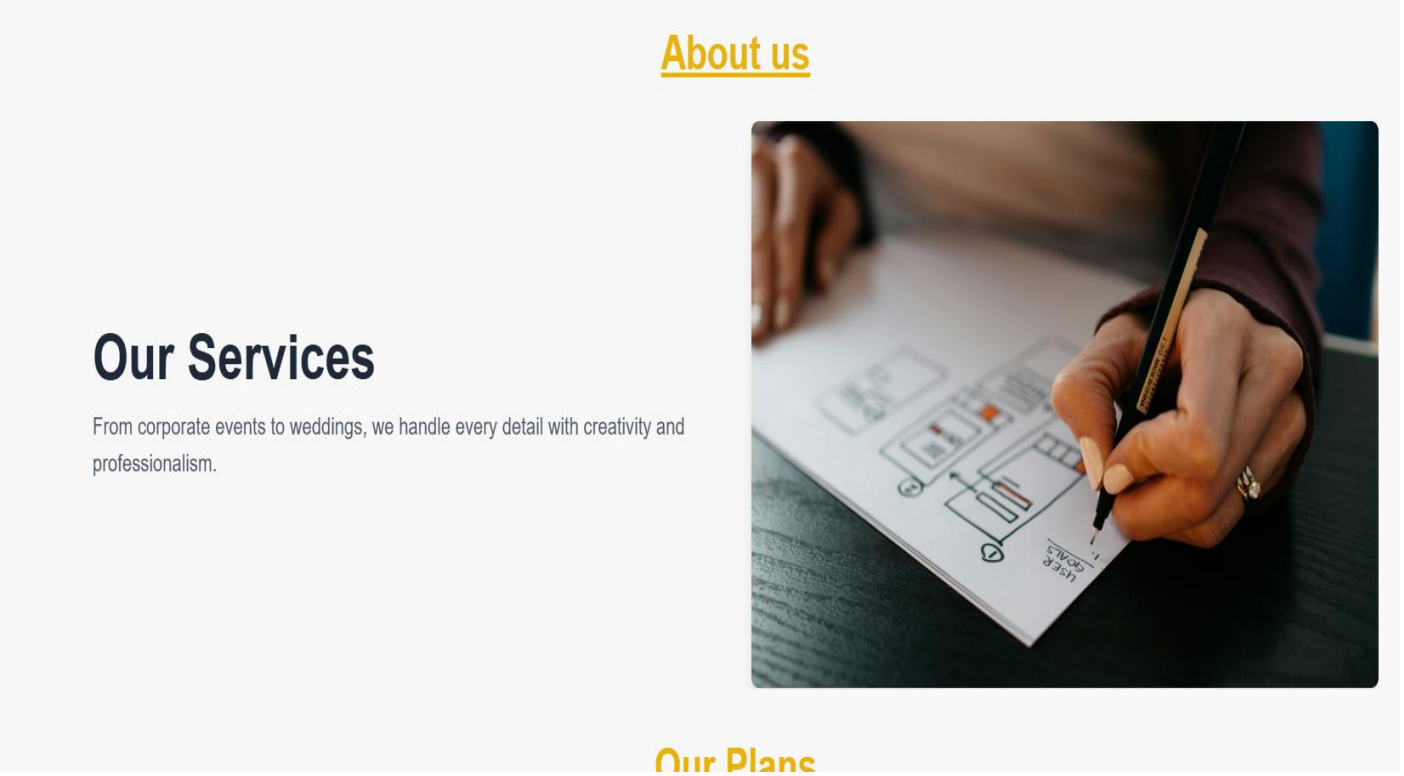
5.3 Results and Screenshots

This section presents the final output of the **EventO** platform. The following screenshots demonstrate the successful integration of the MERN stack, showcasing the responsive user interface and the backend data connectivity.

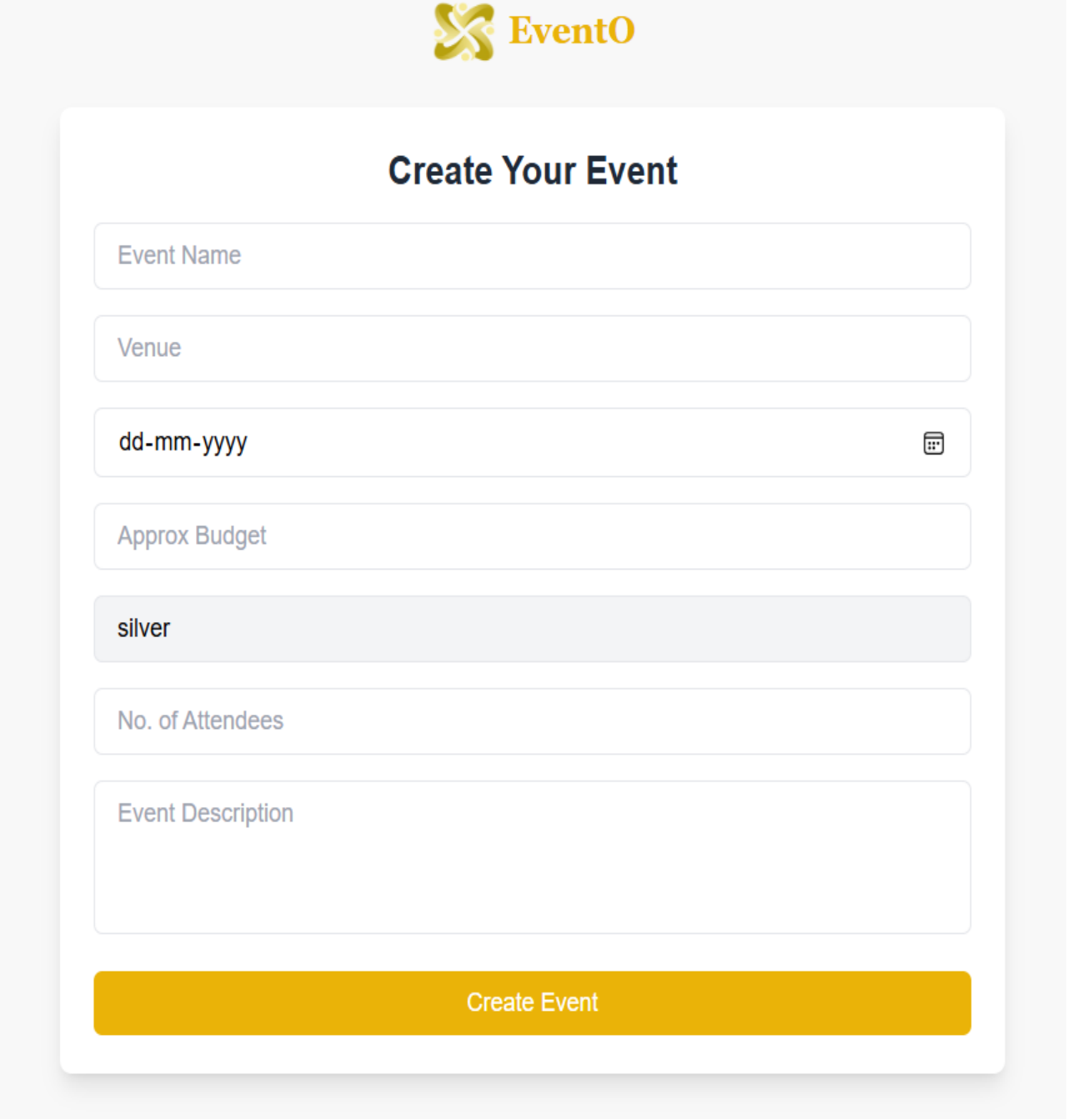
5.3.1 User Authentication (Login/Signup) :-



5.3.2 Home Page and Service Tiers :-

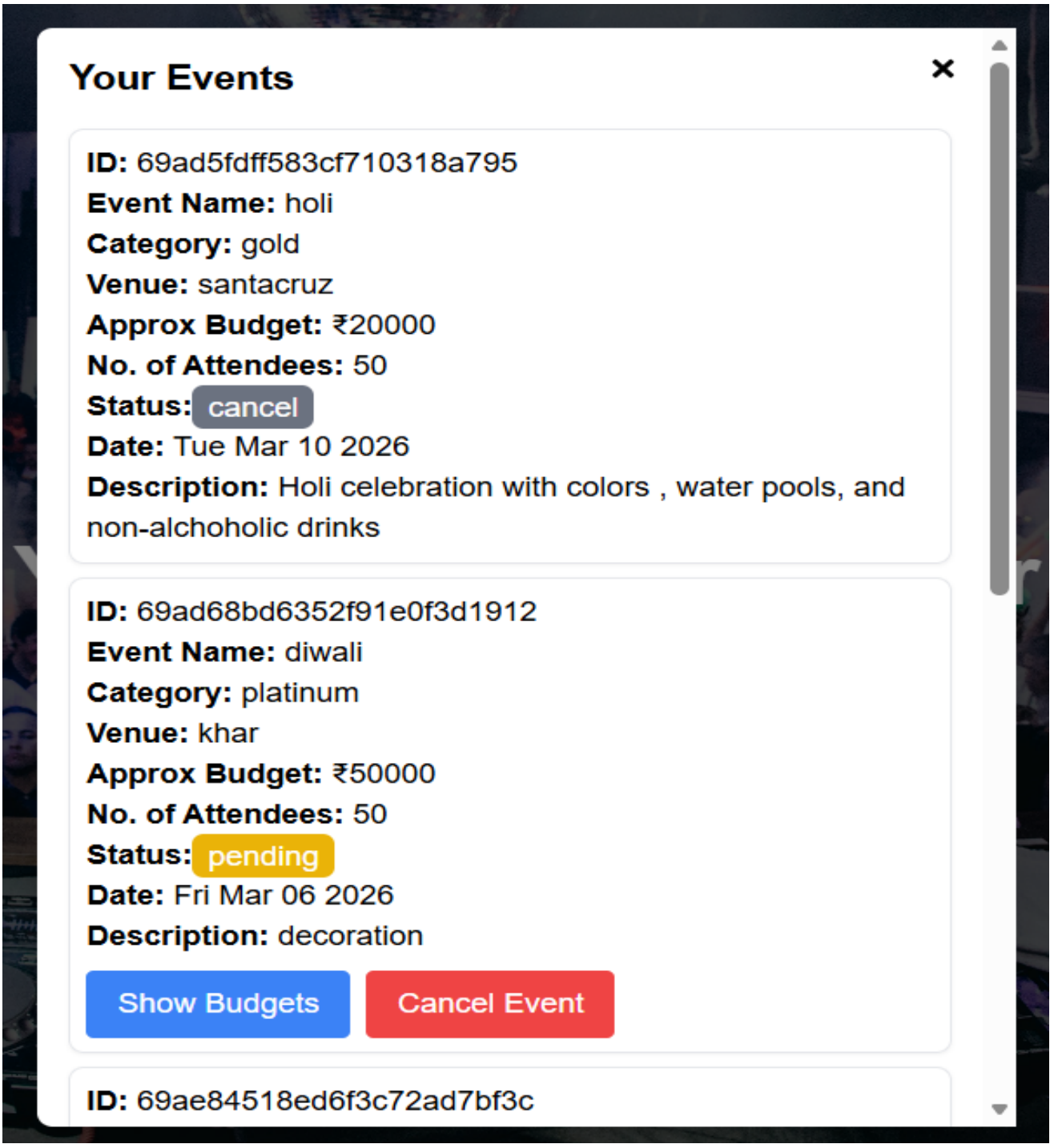


5.3.3 Event Creation Portal :- This interface allows users to input their event requirements. The form captures the Venue, Date, and Budget. Once the "Submit" button is clicked, a POST request is sent to the Express backend, and the data is stored in the MongoDB "Events" collection with a default status of "Pending."



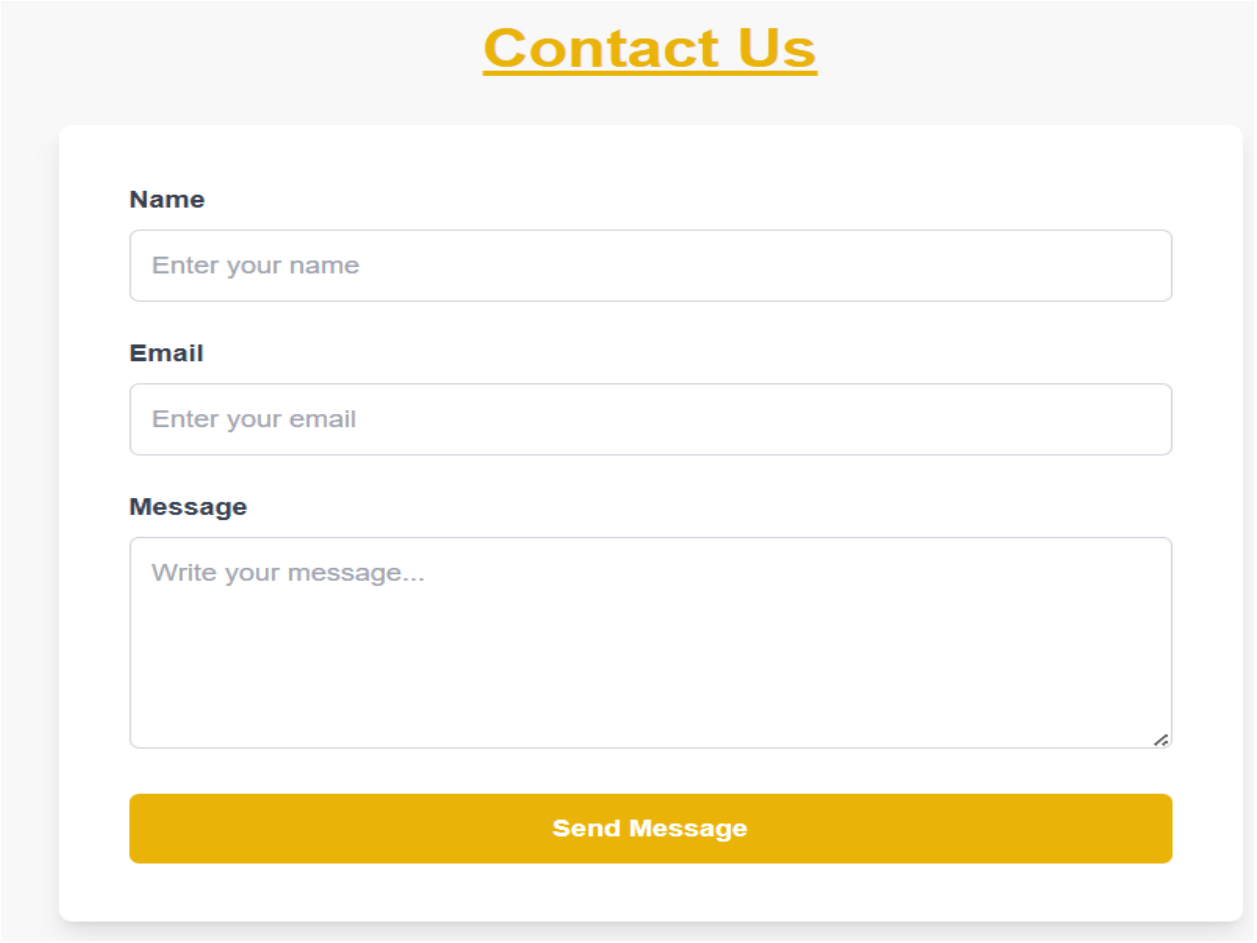
5.3.4 User Dashboard (Tracking) :-

The dashboard provides real-time feedback to the user. This screenshot shows the list of events created by the logged-in user. Each entry displays a unique Tracking ID and the current status (Pending/Confirmed), proving that the frontend is successfully fetching personalized data from the database.

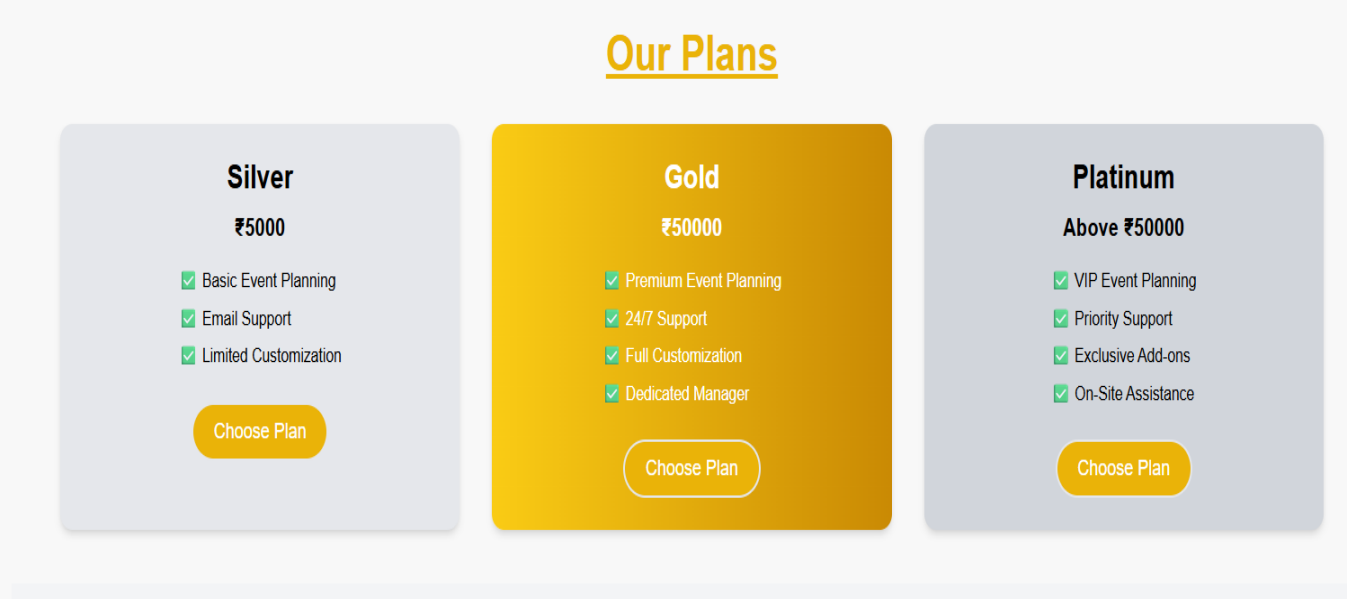


5.3.5 User Feedback and Testimonials :-

This section allows users to submit reviews about their experience with EventO.



5.3.3 Service Plans (Silver, Gold, Platinum) :-



CHAPTER-6

CONCLUSION AND FUTURE WORK

CONCLUSION AND FUTURE WORK

The final chapter of the EventO Black Book summarizes the development journey and outlines the potential for the platform to evolve. As the event management industry shifts toward digital-first solutions, the implementation of this MERN-based system demonstrates how modern web technologies can bridge the gap between complex logistics and user-friendly interfaces. This chapter reflects on the successful realization of the project's objectives and provides a roadmap for future enhancements that could transform EventO into a market-leading enterprise solution.

6.1 Conclusion

The development of EventO has successfully demonstrated the power and efficiency of the MERN Stack in building a scalable, real-time event management platform. By leveraging MongoDB's flexible document model, the system effectively manages diverse event types—ranging from corporate seminars to weddings—without the constraints of a rigid database schema. The React.js frontend provides a seamless, high-performance user experience, ensuring that actions like plan selection and event tracking occur without frustrating page reloads. Throughout the development lifecycle, we adhered to the Waterfall Model, which provided a structured path from requirement analysis to final testing. The result is a robust application that fulfills all functional requirements, including secure user authentication, tiered service selection, and a real-time status dashboard. Ultimately, EventO stands as a professional solution that digitizes the event planning workflow, reducing administrative overhead and increasing transparency for the end-user.

6.2 Future Work

While the current version of **EventO** provides a solid foundation for event coordination, there are several avenues for future expansion to enhance its commercial viability.

- **Integration of a Live Payment Gateway:** In future iterations, we plan to integrate **Stripe** or **Razorpay** APIs. This will allow users to make secure, real-time payments directly for their selected Silver, Gold, or Platinum plans, automating the billing process entirely.
- **AI-Powered Venue Recommendation:** By utilizing **Machine Learning** algorithms, the system could analyze a user's past bookings and budget to suggest the most suitable venues or vendors, providing a highly personalized planning experience.
- **Vendor Management Portal:** Expanding the platform to include a third actor—**Vendors** (Caterers, Photographers, Decorators)—would create a complete ecosystem where organizers can assign tasks and track vendor progress within the same dashboard.
- **Real-time Chat and Notifications:** Implementing **Socket.io** would enable real-time communication between the client and the admin. Additionally, integrating **Push Notifications** or **Email Alerts** (via Nodemailer) would keep users updated on status changes instantly.
- **Mobile Application:** Developing a cross-platform mobile app using **React Native** would allow users to manage their events on the go, utilizing the same Node.js backend already established in this project.

Chapter-7

REFERENCE

References

1. Visual Studio Code Official Website.
   https://code.visualstudio.com/
2. Node.js Documentation.
   https://nodejs.org/
3. Express.js Official Guide.
   https://expressjs.com/
4. MongoDB Documentation.
   https://www.mongodb.com/docs/
5. React Official Documentation.
   https://react.dev/
6. GitHub for project repository and sample codes.
   https://github.com/
7. W3Schools Web Tutorials – HTML, CSS, JavaScript, and PHP Learning Resources.
   https://www.w3schools.com/
8. Mozilla Developer Network (MDN) Web Docs – Web Development Documentation.
   https://developer.mozilla.org/
9. Kurose, J. F., & Ross, K. W. (2017). *Computer Networking: A Top-Down Approach*. Pearson Education.
10. Pressman, R. S. (2014). *Software Engineering: A Practitioner's Approach*. McGraw-Hill Education.